

Ex.no:8	MINI PROJECT – HEALTHCARE MONITORING & PATIENT ANALYTICS

Domain and Problem Statement:

Domain: Healthcare Analytics & Patient Monitoring

Problem Statement: Healthcare monitoring systems generate large volumes of patient vital-sign data such as heart rate, oxygen saturation, blood pressure, body temperature and alert indicators. Manually analyzing these measurements makes it difficult to identify abnormal vital signs, compare patient conditions and understand disease-level health patterns.

This project transforms raw patient monitoring data into a structured analytical solution using Python, MySQL, Power BI and Tableau. The dashboards provide insights into predicted diseases, abnormal vital alerts, fall detection, health alert levels and vital-sign patterns while supporting interactive filtering and patient-level analysis.

Dataset Description:

Dataset Name: Patient Data – Healthcare Monitoring System

Dataset Size: 60,000 patient monitoring records.

Original Columns: 13 | Final Analytical Columns: 18

Granularity: Each row represents a patient monitoring record containing vital signs, alert indicators and predicted health information.

Key Entities & Attributes:

- Patient Information: Patient ID
- Vital Signs: Heart Rate, SpO2, Systolic BP, Diastolic BP, Body Temperature
- Monitoring Alerts: Heart Rate Alert, SpO2 Alert, Blood Pressure Alert, Temperature Alert
- Health Information: Predicted Disease, Health Alert Level
- Data Quality: Data Accuracy, Accuracy Category
- Derived Measures: Pulse Pressure, Mean Arterial Pressure, Abnormal Vital Count
- Monitoring Event: Fall Detection

Data Source:

Primary Source: Kaggle – Patient Healthcare Monitoring Dataset.

Processing: Python / Pandas | Database: MySQL 8.0 – healthcare_analytics | Visualization: Power BI and Tableau Public

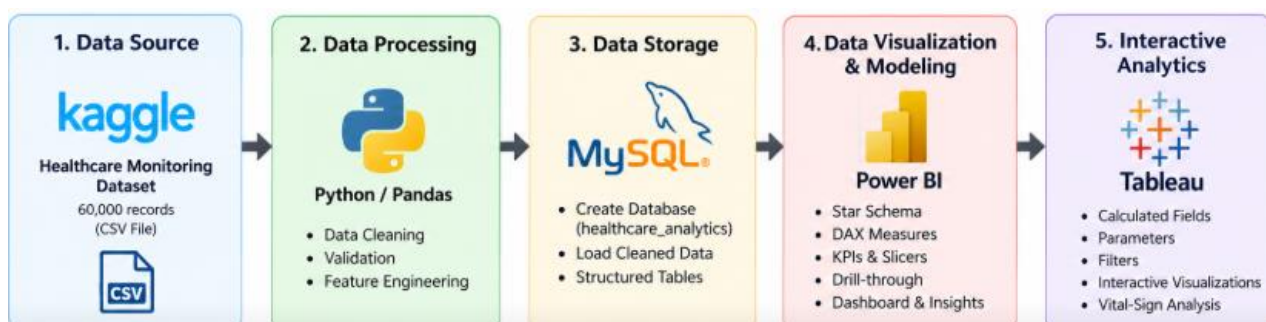


Fig 1: Overall Healthcare Analytics Project Architecture

1. Python – Data Cleaning & Transformation

```
import pandas as pd

# Load dataset
df = pd.read_csv("Synthetic_patient-HealthCare-Monitoring_dataset.csv")

# Create a copy for processing
healthcare = df.copy()

print("Rows:", healthcare.shape[0])
print("Columns:", healthcare.shape[1])
healthcare.head()
```

Standardizing Column Names

```
healthcare.columns = [
    "PatientID", "HeartRate", "SpO2", "SystolicBP",
    "DiastolicBP", "BodyTemperature", "FallDetection",
    "PredictedDisease", "DataAccuracy", "HeartRateAlert",
    "SpO2Alert", "BloodPressureAlert", "TemperatureAlert"
]
```

Data Type Conversion

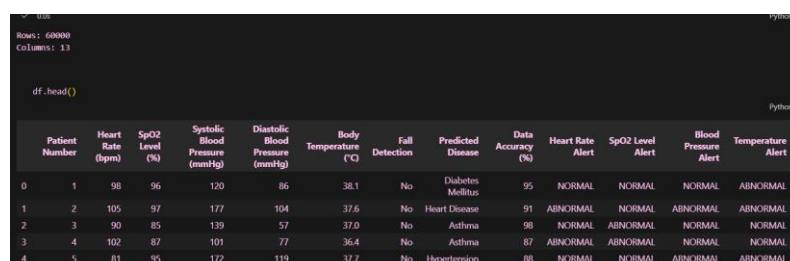
```
numeric_columns = [
    "PatientID", "HeartRate", "SpO2", "SystolicBP",
    "DiastolicBP", "BodyTemperature", "DataAccuracy"
]
```

```
for column in numeric_columns:
    healthcare[column] = pd.to_numeric(
        healthcare[column], errors="coerce"
    )
```

Categorical Data Cleaning

```
categorical_columns = [
    "FallDetection", "PredictedDisease", "HeartRateAlert",
    "SpO2Alert", "BloodPressureAlert", "TemperatureAlert"
]
```

```
for column in categorical_columns:
    healthcare[column] = healthcare[column].astype(str).str.strip()
```



	Patient Number	Heart Rate (bpm)	SpO2 Level (%)	Systolic Blood Pressure (mmHg)	Diastolic Blood Pressure (mmHg)	Body Temperature (°C)	Fall Detection	Predicted Disease	Data Accuracy (%)	Heart Rate Alert	SpO2 Level Alert	Blood Pressure Alert	Temperature Alert
0	1	98	96	120	86	38.1	No	Diabetes Mellitus	95	NORMAL	NORMAL	NORMAL	ABNORMAL
1	2	105	97	177	104	37.6	No	Heart Disease	91	ABNORMAL	NORMAL	ABNORMAL	ABNORMAL
2	3	90	85	139	57	37.0	No	Asthma	98	NORMAL	ABNORMAL	NORMAL	NORMAL
3	4	102	87	101	77	36.4	No	Asthma	87	ABNORMAL	ABNORMAL	NORMAL	NORMAL
4	5	81	95	172	119	37.7	No	Hypertension	88	NORMAL	NORMAL	ABNORMAL	ABNORMAL

Fig 2: Dataset / Power Query Data Preview

Data Validation & Feature Engineering

Data Validation

- Checked the dataset for missing values and duplicate records.
- Verified numeric columns and categorical values for consistent types.
- Reviewed potential outliers using statistical screening.
- Valid observations were retained rather than automatically removed.

Feature Engineering

```
healthcare["PulsePressure"] = (  
    healthcare["SystolicBP"] - healthcare["DiastolicBP"]  
)
```

```
healthcare["MeanArterialPressure"] = (  
    healthcare["SystolicBP"] +  
    2 * healthcare["DiastolicBP"]  
) / 3
```

Abnormal Vital Count

```
alert_columns = [  
    "HeartRateAlert", "SpO2Alert",  
    "BloodPressureAlert", "TemperatureAlert"  
)  
  
healthcare["AbnormalVitalCount"] = (  
    healthcare[alert_columns].eq("ABNORMAL").sum(axis=1)  
)
```

Health Alert Level

```
healthcare["HealthAlertLevel"] = pd.cut(  
    healthcare["AbnormalVitalCount"],  
    bins=[-1, 0, 1, 2, 4],  
    labels=["Normal", "Low", "Moderate", "High"]  
)
```

Accuracy Category

```
healthcare["AccuracyCategory"] = pd.cut(  
    healthcare["DataAccuracy"],  
    bins=[84, 89, 94, 97, 100],  
    labels=["Acceptable", "Good", "Very Good", "Excellent"]  
)
```

Final Dataset: 60,000 rows × 18 analytical columns.

Python – Final Dataset Export & Validation

Export Cleaned Dataset

```
healthcare.to_csv(  
    "Patient_Data_Healthcare_Cleaned.csv",  
    index=False  
)
```

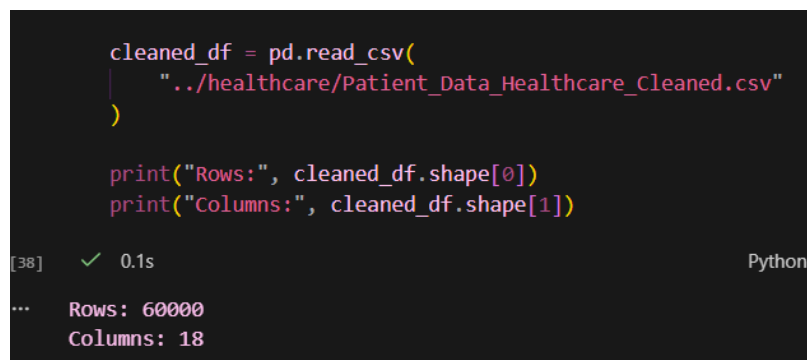
```
print("Cleaned Shape:", healthcare.shape)
```

Final Analytical Columns

PatientID, HeartRate, SpO2, SystolicBP, DiastolicBP, BodyTemperature, FallDetection, PredictedDisease, DataAccuracy, HeartRateAlert, SpO2Alert, BloodPressureAlert, TemperatureAlert, PulsePressure, MeanArterialPressure, AbnormalVitalCount, HealthAlertLevel, AccuracyCategory

Validation Summary

- Original dataset: 60,000 rows and 13 columns.
- Cleaned dataset: 60,000 rows and 18 columns.
- No missing values remained in the analytical dataset.
- No duplicate records remained.
- Derived fields were validated for numerical consistency.
- The cleaned CSV was used as the Tableau Public input.



```
cleaned_df = pd.read_csv(  
    "../healthcare/Patient_Data_Healthcare_Cleaned.csv"  
)  
  
print("Rows:", cleaned_df.shape[0])  
print("Columns:", cleaned_df.shape[1])
```

[38] ✓ 0.1s Python

... Rows: 60000
Columns: 18

Fig3: Dataset has been cleaned

2. MySQL – Database and Data Loading

Create Database

```
CREATE DATABASE healthcare_analytics;  
USE healthcare_analytics;
```

Create Patient Monitoring Table

```
CREATE TABLE patient_monitoring (  
    PatientID INT,  
    HeartRate INT,  
    SpO2 INT,  
    SystolicBP INT,  
    DiastolicBP INT,  
    BodyTemperature DECIMAL(4,1),  
    FallDetection VARCHAR(10),  
    PredictedDisease VARCHAR(100),  
    DataAccuracy INT,  
    HeartRateAlert VARCHAR(20),  
    SpO2Alert VARCHAR(20),  
    BloodPressureAlert VARCHAR(20),  
    TemperatureAlert VARCHAR(20),  
    PulsePressure INT,  
    MeanArterialPressure DECIMAL(6,2),  
    AbnormalVitalCount INT,  
    HealthAlertLevel VARCHAR(20),  
    AccuracyCategory VARCHAR(20)  
);
```

Python MySQL Connection & Bulk Insert

```
import mysql.connector  
  
connection = mysql.connector.connect(  
    host="localhost",  
    user="root",  
    password="YOUR_MYSQL_PASSWORD",  
    database="healthcare_analytics"  
)  
cursor = connection.cursor()  
# Insert cleaned records in batches  
for i in range(0, len(data), 5000):  
    cursor.executemany(insert_sql, data[i:i+5000])  
    connection.commit()
```

The cleaned dataset was loaded in batches of 5,000 records. The final MySQL verification confirmed 60,000 records in patient_monitoring.

```
cursor.close()  
connection.close()  
  
print("MySQL connection closed successfully!")  
  
Python  
MySQL connection closed successfully!
```

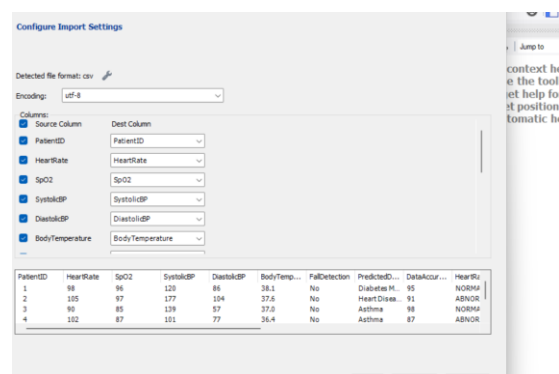


Fig 4& 5: MYSQL is connected successfully

Procedure

Step 1: Data Preparation & Preprocessing

- Load the raw healthcare monitoring dataset using Pandas.
- Inspect structure, missing values, duplicates and data types.
- Standardize column names and categorical values.
- Perform numerical validation and retain valid observations.

Step 2: Feature Engineering

- Calculate Pulse Pressure and Mean Arterial Pressure.
- Count abnormal vital-sign alerts.
- Classify Health Alert Level as Normal, Low, Moderate or High.
- Categorize Data Accuracy into four analytical groups.

Step 3: SQL Database Connection & Data Ingestion

- Create healthcare_analytics in MySQL 8.0.
- Create the patient_monitoring table.
- Insert the cleaned CSV data using Python MySQL Connector.
- Verify the final record count.

Step 4: Power BI Analytics

- Connect Power BI to MySQL.
- Create a star-schema-style model.
- Develop DAX measures, KPI cards, charts and slicers.
- Implement patient-level drill-through.

Step 5: Tableau Analytics

- Connect Tableau Public to the cleaned CSV.
- Create worksheets and calculated fields.
- Develop the Vital Sign Selector parameter.
- Apply disease and monitoring filters.

3. Power BI – Data Connection & Preparation

Connecting Power BI to MySQL

- Open Power BI Desktop.
- Select Get Data → MySQL Database.
- Enter Server: localhost and Database: healthcare_analytics.
- Authenticate using MySQL credentials.
- Select patient_monitoring and load the dataset.

Power Query Preparation

The imported table contains 60,000 records and 18 analytical fields, including numerical vital signs, alert indicators, disease classifications and engineered health metrics.

Main Power BI Table

FactPatientMonitoring

The imported table was renamed to FactPatientMonitoring to represent the central fact table in the analytical model.

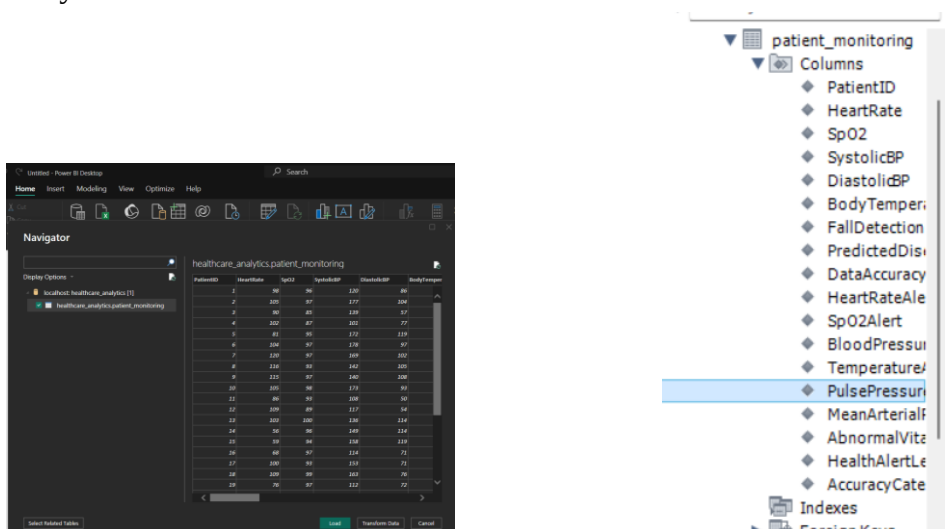


Fig 6&7: Connecting MYSQL with POWER BI and loaded the data

Step 3: Star Schema Data Modeling

Fact Table – FactPatientMonitoring

Contains patient monitoring measurements, alert fields, predicted disease, data quality and derived health indicators.

Dimension Tables

- DimPatient – PatientID
- DimDisease – PredictedDisease
- DimHealthAlert – HealthAlertLevel
- DimAccuracy – AccuracyCategory
- DimFallDetection – FallDetection

Relationships

Each dimension table is connected to FactPatientMonitoring through its corresponding analytical field using one-to-many, single-direction relationships.

```
1 DimPatient =
2 DISTINCT(
3     SELECTCOLUMNS(
4         'healthcare_analytics patient_monitoring',
5         "PatientID", 'healthcare_analytics patient_monitoring'
6         [PatientID]
7     )
8 )

1 DimHealthAlert =
2 DISTINCT(
3     SELECTCOLUMNS(
4         FactPatientMonitoring,
5         "HealthAlertLevel",
6         FactPatientMonitoring[HealthAlertLevel]
7     )
8 )
```

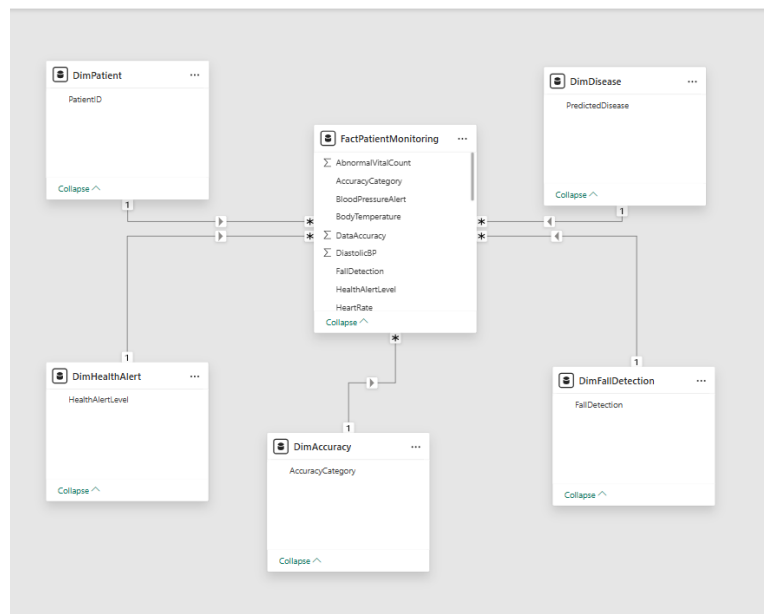


Fig 8: Power BI star-schema-style data model

Step 4: Core Calculations & Measures (DAX)

Power BI Measures

```
Total Patients =  
DISTINCTCOUNT(FactPatientMonitoring[PatientID])  
  
Average Heart Rate =  
AVERAGE(FactPatientMonitoring[HeartRate])  
  
Average SpO2 =  
AVERAGE(FactPatientMonitoring[SpO2])  
  
Average Systolic BP =  
AVERAGE(FactPatientMonitoring[SystolicBP])  
  
Average Diastolic BP =  
AVERAGE(FactPatientMonitoring[DiastolicBP])  
  
Average Body Temperature =  
AVERAGE(FactPatientMonitoring[BodyTemperature])  
  
Patients with Alerts =  
CALCULATE(  
    DISTINCTCOUNT(FactPatientMonitoring[PatientID]),  
    FactPatientMonitoring[AbnormalVitalCount] > 0  
)  
  
Alert Rate =  
DIVIDE([Patients with Alerts], [Total Patients], 0)  
  
Fall Cases =  
CALCULATE(  
    DISTINCTCOUNT(FactPatientMonitoring[PatientID]),  
    FactPatientMonitoring[FallDetection] = "Yes"  
)
```

Additional measures created: Average MAP and Abnormal Vital Alerts

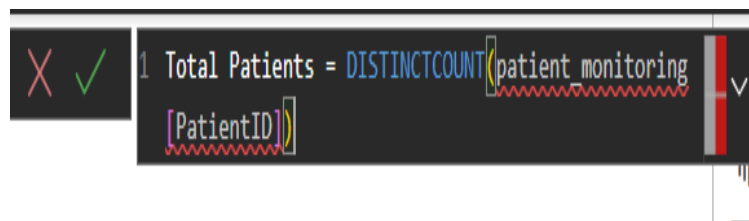
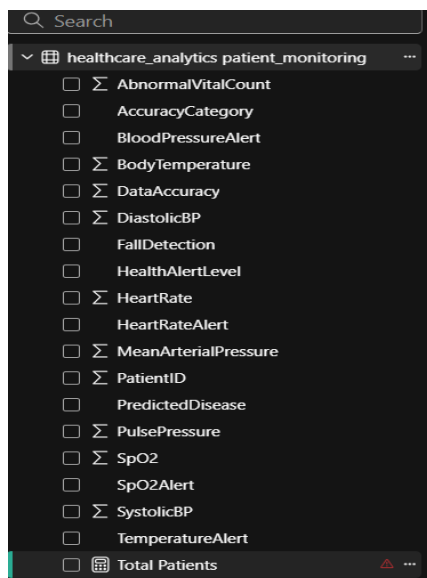


Fig 9: Power BI calculated additional measures with DAX

Step 5: Power BI Final Executive Dashboard

KPI Cards

Total Patients: 60K | Patients with Alerts: 47K | Average SpO2: 94.55 | Average Heart Rate: 94.57 | Average Systolic BP: 139.15

Dashboard Visualizations

- Patients by Predicted Disease – compares disease categories.
- Patients by Accuracy Category – shows data-quality distribution.
- Patients by Fall Detection – compares Yes and No classifications.
- Heart Rate vs SpO2 – scatter analysis of patient vital-sign observations.
- Health Alert Level Distribution – Normal, Low, Moderate and High.
- Health Alert Level by Predicted Disease – disease-wise alert distribution.

Interactive Slicers

Predicted Disease | Health Alert Level | Fall Detection | Data Accuracy



Fig 10: Power BI's Interactive Slicers



Fig 11: Power BI- KPI Cards

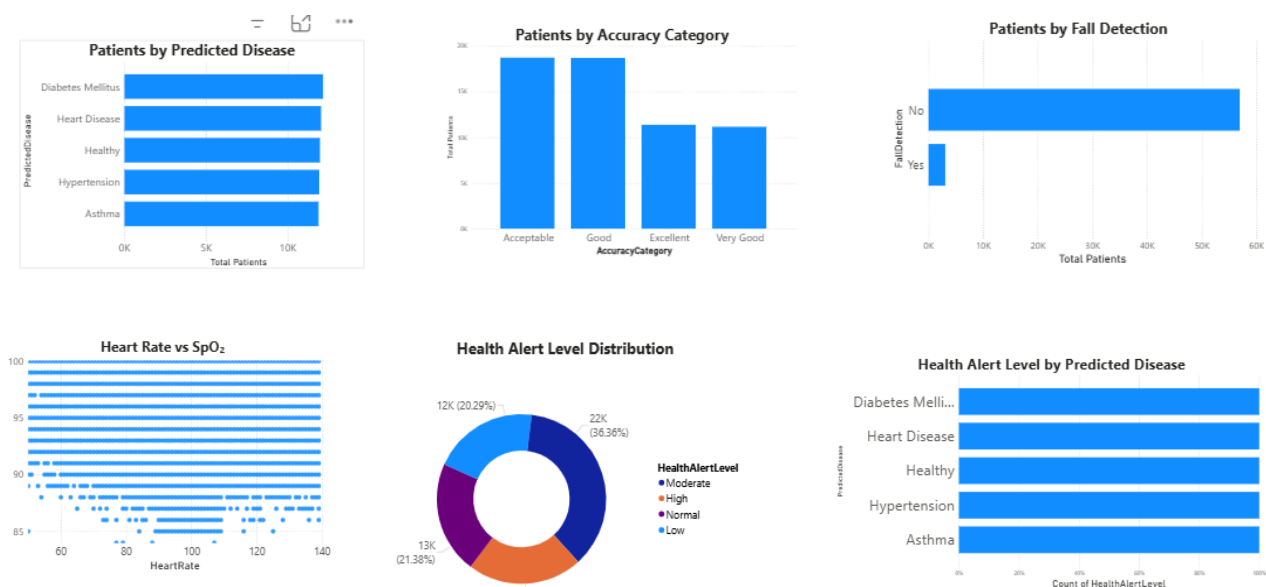


Fig 12: Power BI Healthcare Monitoring dashboard visualization

Power BI Dashboard – Analytical View

The executive dashboard combines KPI cards, categorical comparisons and vital-sign analysis on a single reporting canvas. Users can filter the complete report using the disease, alert-level, fall-detection and accuracy slicers.

Key Dashboard Components

- KPI cards summarize the overall patient monitoring population.
- Disease distribution highlights the number of records associated with each predicted disease.
- Accuracy and fall-detection charts provide data-quality and safety context.
- The Heart Rate vs SpO2 scatter plot supports patient-level pattern exploration.
- The alert-level donut and disease-wise stacked chart show the severity structure of the monitoring data.

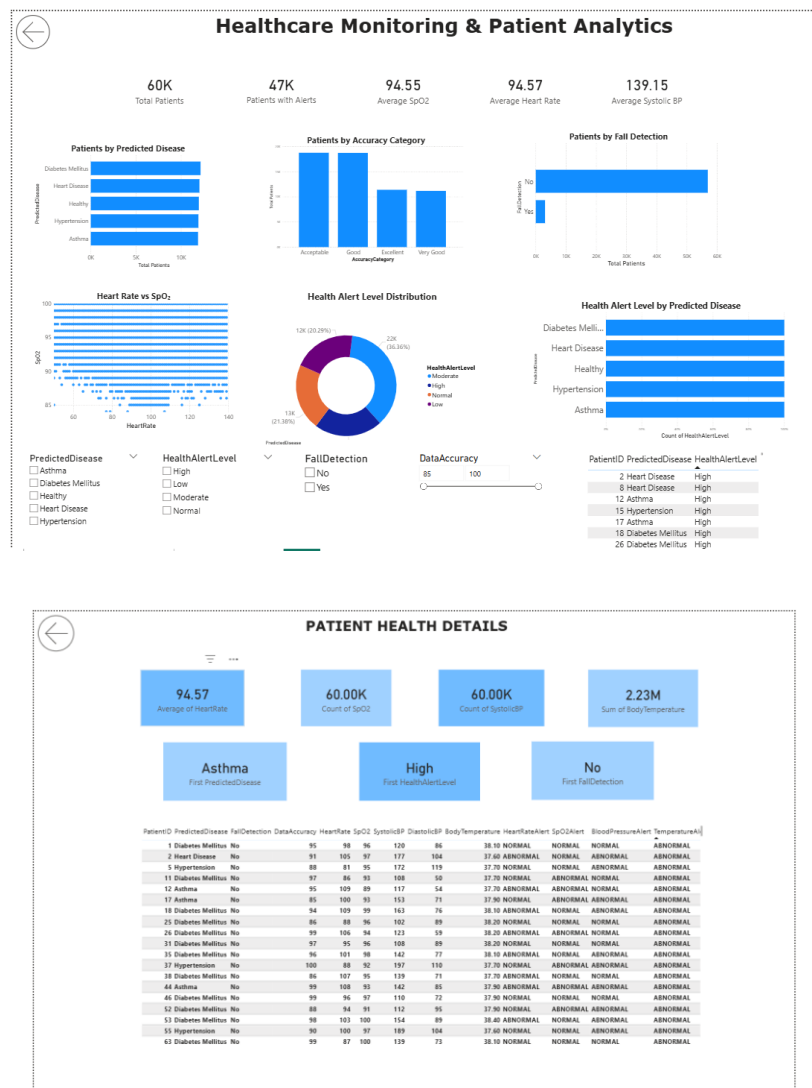


Fig 13: Power BI analytical dashboard view

Power BI Patient-Level Drill-Through

Drill-Through Configuration

A second report page named Patient Details was created. PatientID was added to the Drill-through field and configured as Used as category so that a selected patient can be passed from the executive dashboard to the detailed page.

Patient Details Page

- Heart Rate
- SpO2
- Systolic BP
- Body Temperature
- Predicted Disease
- Health Alert Level
- Fall Detection

The detailed table additionally contains PatientID, Data Accuracy and individual alert fields. A Back button was added to return to the main dashboard.



Fig 8: PatientID drill-through configuration

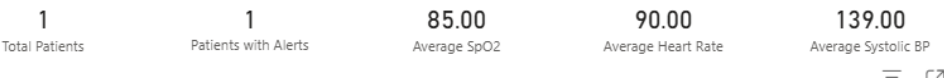


Fig 9: Patient Details with drill-through using Patient's ID

Tableau – Data Connection & Preparation

Tableau Public Data Connection

Tableau Public was used for the visualization stage. The cleaned file Patient_Data_Healthcare_Cleaned.csv was connected through the file-based data source available in Tableau Public.

The cleaned dataset contains 60,000 rows and 18 analytical fields.

Prepared Fields

Patient ID, Heart Rate, SpO2, Systolic BP, Diastolic BP, Body Temperature, Fall Detection, Predicted Disease, Data Accuracy, alert fields, Pulse Pressure, Mean Arterial Pressure, Abnormal Vital Count, Health Alert Level and Accuracy Category.

MySQL was used as the centralized database layer for the Power BI workflow; Tableau Public consumed the cleaned CSV for visualization.

The screenshot shows the Tableau Public interface. At the top, the data source is 'Patient_Data_Healthcare_Cleaned' with 18 fields and 60,000 rows. Below this, a table preview is displayed with 15 rows and 8 columns. The columns are: Patient ID, Heart Rate, SpO2, Systolic BP, Diastolic BP, Body Temp, Fall Detect, and Predicted Disease. The data shows various patient records with their corresponding vital signs and predicted diseases.

Patient ID	Heart Rate	SpO2	Systolic BP	Diastolic BP	Body Temp	Fall Detect	Predicted Disease
1	98	96	120	86	38.10000	No	Diabetes Mellitus
2	105	97	177	104	37.60000	No	Heart Disease
3	90	85	139	57	37.00000	No	Asthma
4	102	87	101	77	36.40000	No	Asthma
5	81	95	172	119	37.70000	No	Hypertension
6	104	97	178	97	36.90000	No	Heart Disease
7	120	97	169	102	36.90000	No	Heart Disease
8	116	93	142	105	37.30000	No	Heart Disease
9	115	97	140	108	37.50000	No	Heart Disease
10	105	98	173	93	37.00000	No	Heart Disease
11	86	93	108	50	37.70000	No	Diabetes Mellitus
12	109	89	117	54	37.70000	No	Asthma
13	103	100	136	114	37.20000	No	Diabetes Mellitus
14	56	96	149	114	36.50000	No	Hypertension
15	59	94	158	119	36.00000	No	Hypertension

Fig 10: Tableau Public connection and data loaded with SQL connector

Tableau Worksheets – Core Visualizations

1. Patient Distribution by Predicted Disease

The bar chart compares patient records across Asthma, Diabetes Mellitus, Healthy, Heart Disease and Hypertension.

2. Heart Rate Alert Distribution

Normal and abnormal heart-rate alerts are compared using the monitoring alert field.

3. Heart Rate vs SpO2

A scatter plot is used to examine the distribution of Heart Rate and SpO2 observations, with Patient ID on Detail and Predicted Disease used for comparison.

4. Health Alert Level Distribution

The visualization shows the number of records classified as Normal, Low, Moderate and High.

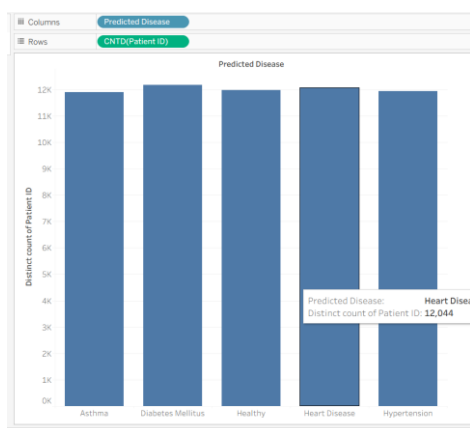


Fig 11: Tableau Predicted Disease distribution

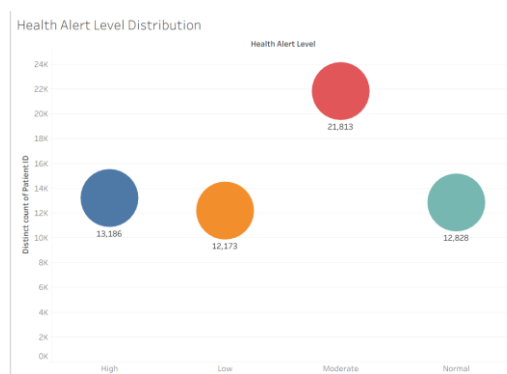


Fig 12: Tableau Health Alert Level distribution



Fig 13: Tableau Heart Rate vs SpO2 analysis

Tableau – Advanced Visualization & Calculated Fields

Calculated Field: Average Heart Rate

AVG([Heart Rate])

Selected Vital Sign Calculated Field

```
CASE [Vital Sign Selector]
WHEN "Heart Rate" THEN [Heart Rate]
WHEN "SpO2" THEN [SpO2]
WHEN "Systolic BP" THEN [Systolic BP]
WHEN "Body Temperature" THEN [Body Temperature]
END
```

The calculated field makes the visualization responsive to the selected vital-sign parameter and allows one analytical view to switch between multiple patient measurements.

Health Alert Level by Predicted Disease

A stacked bar visualization was created to compare Normal, Low, Moderate and High alert levels across the predicted disease categories.

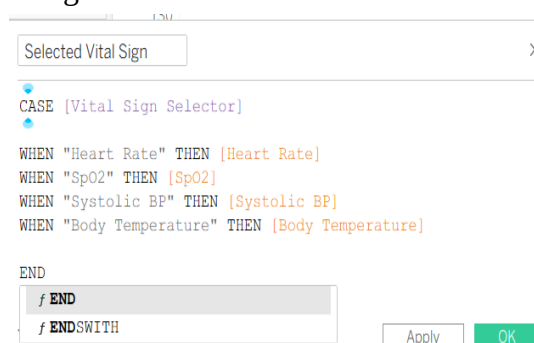


Fig 14: Tableau calculated-field

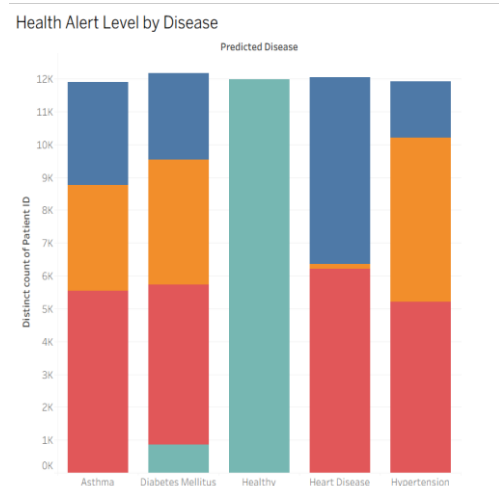
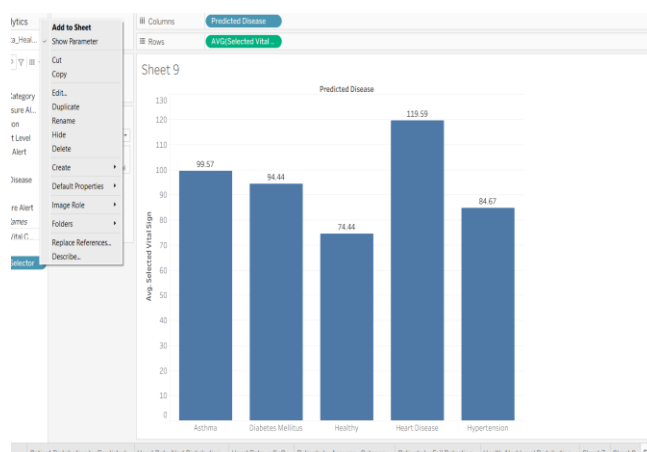


Fig 15& 16: Tableau calculated-field's analysis

Tableau – Parameters & Filters

Vital Sign Selector Parameter

Parameter Name: Vital Sign Selector | Data Type: String | Default: Heart Rate

Allowed Values: Heart Rate, SpO2, Systolic BP, Body Temperature

Dynamic Parameter Use

The parameter controls the Selected Vital Sign calculated field. Changing the parameter changes the measure displayed in the analytical worksheet.

Predicted Disease Filter

A Predicted Disease filter was added to allow users to focus the analysis on Asthma, Diabetes Mellitus, Healthy, Heart Disease or Hypertension.

Other Analytical Filters

- Health Alert Level
- Fall Detection
- Data Accuracy / Accuracy Category

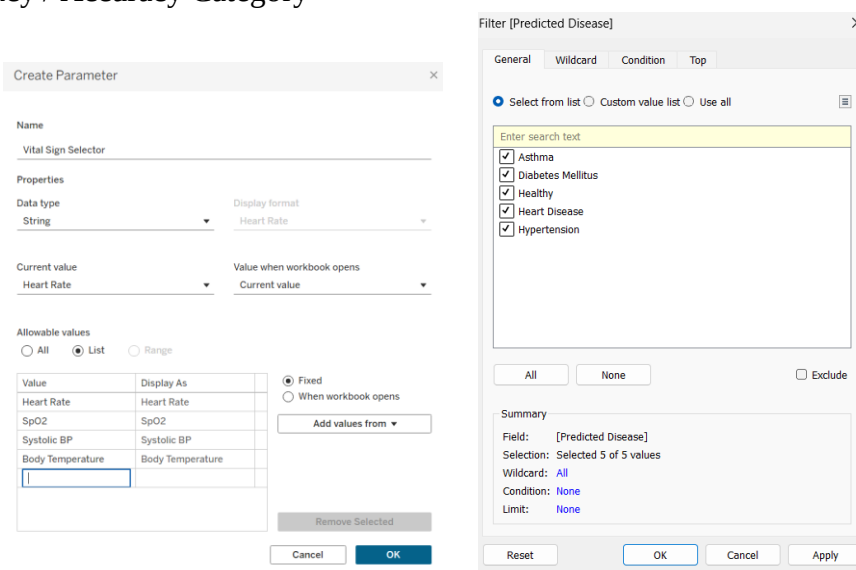


Fig 17 & 18: Tableau Vital Sign Selector parameter and Filtering using Predicted Disease



Fig 19: Tableau Accuracy Category analysis

Tableau – Patient-Level Analysis

A detailed Tableau worksheet was created using Patient ID as the row-level dimension. Vital-sign measures and patient classification fields are displayed together for granular analysis.

Patient-Level Measures

- Average Heart Rate
- Average SpO2
- Average Systolic BP
- Average Diastolic BP
- Average Body Temperature

Patient Classification

- Predicted Disease
- Health Alert Level
- Fall Detection
- Data Accuracy

This patient-level view complements the aggregate dashboard by allowing individual monitoring records to be inspected alongside their vital-sign measurements and health status classifications.



Fig 20: Tableau patient-level monitoring worksheet

Observed Insights & Conclusion

Observed Insights

- The project analyzes 60,000 patient monitoring records.
- Predicted disease volumes are relatively balanced; Diabetes Mellitus has the highest observed count at 12,167 records.
- 47,172 patients have at least one abnormal vital sign, representing approximately 78.62% of the dataset.
- The dataset contains 97,052 abnormal vital-sign alerts across the four monitored vital categories.
- Heart Rate alerts include 23,008 abnormal records and 36,992 normal records.
- Moderate is the largest Health Alert Level category with 21,813 records.
- Fall detection is predominantly classified as No, with 56,926 records, while 3,074 are classified as Yes.
- Average monitoring values include Heart Rate 94.57 bpm, SpO2 94.55%, Systolic BP 139.15 mmHg and Body Temperature 37.12°C.

Conclusion

The project demonstrates an end-to-end healthcare analytics workflow in which Python performs data preparation and feature engineering, MySQL provides structured storage, Power BI provides the primary interactive dashboard and drill-through analysis, and Tableau Public provides complementary parameter-driven visual analytics.

The resulting solution supports disease comparison, alert monitoring, data-quality analysis, fall-detection review and patient-level exploration through interactive business-intelligence techniques.

Project Deliverables

Dataset | Python Notebook | Cleaned CSV | MySQL Database | Power BI Dashboard (.pbix) | Tableau Public Worksheets | DAX Measures | Calculated Fields | Parameter | Filters | Drill-through | Documentation

Technologies Used: Python | Pandas | MySQL 8.0 | Power BI | DAX | Tableau Public